Web Client Setup Guide

Overview

The cloud gaming web client provides React components that can be integrated into any React application to enable cloud gaming streaming.

Installation

Option 1: Use as Standalone App

Run the example application:

```
cd web-client
npm install
npm run dev
```

The app will be available at `http://localhost:5173`

Option 2: Integrate Into Your React App

Install the package (or copy the `src` folder):

```
# If published to npm
npm install cloud-gaming-web-client

# Or copy source files
cp -r web-client/src/* your-app/src/cloud-gaming/
```

Basic Integration

1. Simple Integration with StreamPlayer Component

```
import { StreamPlayer } from 'cloud-gaming-web-client';
// Or: import { StreamPlayer } from './cloud-gaming/components/StreamPlayer';

function App() {
  const [sessionId, setSessionId] = useState('');

  return (
    <div>
      <input
        type="text"
        value={sessionId}
        onChange={(e) => setSessionId(e.target.value)}
        placeholder="Enter Session ID"
      />

      <StreamPlayer
        signalingServerUrl="https://signaling.yourdomain.com"
        sessionId={sessionId}
        autoConnect={false}
        showControls={true}
        showStats={true}
        onConnect={() => console.log('Connected!')}
        onDisconnect={() => console.log('Disconnected!')}
        onError={(error) => console.error('Error:', error)}
      />
    </div>
  );
}
```

2. Advanced Integration with Hooks

For more control, use the hooks directly:

```
import { useStreamConnection } from 'cloud-gaming-web-client';
import { InputHandler } from 'cloud-gaming-web-client';

function CustomStreamingComponent() {
  const {
    connected,
    connecting,
    error,
    sessionInfo,
    stats,
    videoRef,
    connect,
    disconnect,
    sendInput,
    changeQuality,
  } = useStreamConnection({
    signalingServerUrl: 'https://signaling.yourdomain.com',
    sessionId: 'your-session-id',
    autoConnect: false,
  });

  return (
    <div>
      <video ref={videoRef} autoPlay playsInline />

      <button onClick={connect} disabled={connected || connecting}>
        {connecting ? 'Connecting...' : 'Connect'}
      </button>

      <button onClick={disconnect} disabled={!connected}>
        Disconnect
      </button>

      {connected && (
        <InputHandler
          enabled={true}
          onInput={sendInput}
          showGamepadStatus={true}
        />
      )}

      {stats && (
        <div>
          <p>Latency: {stats.latency}ms</p>
          <p>FPS: {stats.frameRate}</p>
          <p>Bitrate: {stats.bitrate} kbps</p>
        </div>
      )}
    </div>
  );
}
```

3. Controller Detection

Use the `useController` hook for gamepad support:

```tsx
import { useController } from 'cloud-gaming-web-client';

function GamepadIndicator() {
  const { gamepads, connected, activeGamepad } = useController();

  return (
    <div>
      {connected ? (
        <div>
          <p>✓ Gamepad Connected</p>
          <p>ID: {activeGamepad?.id}</p>
          <p>Buttons: {activeGamepad?.buttons.length}</p>
          <p>Axes: {activeGamepad?.axes.length}</p>
        </div>
      ) : (
        <p>No gamepad detected. Connect a controller and press any button.</p>
      )}
    </div>
  );
}
```

Components API

StreamPlayer

```tsx
interface StreamPlayerProps {
  signalingServerUrl: string;     // Signaling server URL
  sessionId: string;               // VM session ID
  autoConnect?: boolean;           // Auto-connect on mount (default: false)
  showControls?: boolean;          // Show quality/connect controls (default: true)
  showStats?: boolean;             // Show network stats overlay (default: true)
  className?: string;              // Custom CSS class
  onConnect?: () => void;          // Called when connected
  onDisconnect?: () => void;       // Called when disconnected
  onError?: (error: string) => void; // Called on error
}
```

InputHandler

```tsx
interface InputHandlerProps {
  enabled: boolean;                       // Enable/disable input capture
  onInput: (inputData: any) => void;     // Input event callback
  captureElement?: React.RefObject<HTMLElement>; // Element to capture from
  showGamepadStatus?: boolean;           // Show gamepad connection status
}
```

Hooks API

useStreamConnection

```
const {
  connected,      // boolean: Connection status
  connecting,     // boolean: Connecting in progress
  error,          // string | null: Error message
  sessionInfo,    // SessionInfo | null: VM session details
  stats,          // ConnectionStats | null: Network statistics
  videoRef,       // RefObject<HTMLVideoElement>: Video element ref
  connect,        // () => Promise<void>: Connect to stream
  disconnect,     // () => void: Disconnect from stream
  sendInput,      // (data: any) => void: Send input to VM
  changeQuality,  // (quality: 'high'|'medium'|'low') => void: Change quality
} = useStreamConnection({
  signalingServerUrl: string,
  sessionId: string,
  autoConnect?: boolean,
});
```

useController

```
const {
  gamepads,       // GamepadState[]: Array of connected gamepads
  connected,      // boolean: At least one gamepad connected
  activeGamepad,  // GamepadState | null: Primary gamepad (index 0)
} = useController();
```

Styling

Custom Styling

Override default styles:

```
/* Override StreamPlayer styles */
.stream-player {
  max-width: 1920px;
  border-radius: 12px;
}

.stream-player .video-element {
  border: 2px solid #667eea;
}

.stream-player .stats-overlay {
  background: rgba(0, 0, 0, 0.9);
  font-size: 14px;
}

/* Quality buttons */
.stream-player .quality-controls button.active {
  background: #667eea;
}
```

Theming

Create a theme file:

```
:root {
  --cg-primary-color: #667eea;
  --cg-background-color: #000;
  --cg-text-color: #fff;
  --cg-error-color: #ff4444;
}
```

Configuration

Environment Variables

Create `.env` file:

```
VITE_SIGNALING_SERVER_URL=https://signaling.yourdomain.com
VITE_DEFAULT_SESSION_ID=
VITE_ENABLE_STATS=true
```

Use in code:

```
const signalingUrl = import.meta.env.VITE_SIGNALING_SERVER_URL;
```

Deployment

Build for Production

```
npm run build
```

Output will be in `dist/` directory.

Deploy to Static Hosting

Vercel

```
npm install -g vercel
vercel
```

Netlify

```
npm install -g netlify-cli
netlify deploy --prod
```

AWS S3 + CloudFront

```
aws s3 sync dist/ s3://your-bucket-name/
aws cloudfront create-invalidation --distribution-id YOUR_DIST_ID --paths "/*"
```

Nginx

```
server {
    listen 80;
    server_name gaming.yourdomain.com;
    root /var/www/cloud-gaming-client;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    # CORS headers for WebRTC
    add_header Access-Control-Allow-Origin *;
    add_header Access-Control-Allow-Methods "GET, POST, OPTIONS";
}
```

Browser Compatibility

Supported Browsers

- ✅ Chrome 90+
- ✅ Firefox 88+
- ✅ Safari 14.1+
- ✅ Edge 90+
- ❌ IE 11 (not supported)

Feature Detection

```
function checkBrowserSupport() {
  const hasWebRTC = !!(navigator.mediaDevices && RTCPeerConnection);
  const hasGamepad = !!navigator.getGamepads;
  const hasPointerLock = 'pointerLockElement' in document;

  if (!hasWebRTC) {
    alert('Your browser does not support WebRTC');
  }

  return { hasWebRTC, hasGamepad, hasPointerLock };
}
```

Advanced Features

Custom Input Mapping

```
function CustomInputHandler() {
  const handleInput = (inputData: any) => {
    // Custom input processing
    if (inputData.type === 'gamepad') {
      // Map gamepad buttons to custom actions
      const customMapping = {
        0: 'jump',
        1: 'crouch',
        2: 'interact',
        3: 'reload',
      };

      const action = customMapping[inputData.button];
      sendInput({ type: 'action', action });
    } else {
      sendInput(inputData);
    }
  };

  return <InputHandler enabled={true} onInput={handleInput} />;
}
```

Network Quality Indicator

```
function QualityIndicator({ stats }: { stats: ConnectionStats }) {
  const getQualityLevel = () => {
    if (!stats) return 'unknown';
    if (stats.latency < 50 && stats.packetLoss < 1) return 'excellent';
    if (stats.latency < 100 && stats.packetLoss < 3) return 'good';
    if (stats.latency < 150 && stats.packetLoss < 5) return 'fair';
    return 'poor';
  };

  const quality = getQualityLevel();
  const colors = {
    excellent: 'green',
    good: 'yellow',
    fair: 'orange',
    poor: 'red',
    unknown: 'gray',
  };

  return (
    <div style={{ color: colors[quality] }}>
      Connection: {quality.toUpperCase()}
    </div>
  );
}
```

Full-Screen Mode

```
function FullScreenButton() {
  const videoRef = useRef<HTMLVideoElement>(null);

  const toggleFullScreen = () => {
    if (videoRef.current) {
      if (document.fullscreenElement) {
        document.exitFullscreen();
      } else {
        videoRef.current.requestFullscreen();
      }
    }
  };

  return (
    <button onClick={toggleFullScreen}>
      Toggle Fullscreen
    </button>
  );
}
```

Troubleshooting

Video Not Playing

```
// Auto-play might be blocked by browser
// Add user interaction to start video
videoRef.current?.play().catch(err => {
  console.log('Autoplay blocked, user interaction required');
});
```

WebRTC Connection Failed

```
// Check STUN server accessibility
const checkSTUN = async () => {
  const pc = new RTCPeerConnection({
    iceServers: [{ urls: 'stun:stun.l.google.com:19302' }]
  });

  pc.onicecandidate = (event) => {
    if (event.candidate) {
      console.log('STUN server accessible:', event.candidate);
    }
  };

  pc.createDataChannel('test');
  await pc.createOffer();
};
```

High Latency

- Use wired connection instead of WiFi
- Close other network-heavy applications
- Choose lower quality preset
- Check for server/VM location (use closest region)

Performance Optimization

Reduce Bundle Size

```
// vite.config.ts
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
          socketio: ['socket.io-client'],
        },
      },
    },
  },
});
```

Lazy Loading

```
import { lazy, Suspense } from 'react';

const StreamPlayer = lazy(() => import('./components/StreamPlayer'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <StreamPlayer {...props} />
    </Suspense>
  );
}
```

Next Steps

1. Test with your gaming VM
2. Customize styling for your brand
3. Add analytics and monitoring
4. Implement additional features (recording, chat, etc.)
5. Deploy to production